

UNIVERSIDAD DEL VALLE

Software Quality Assurance

Plan de aseguramiento y pruebas de UniFood

Integración de la calidad con Scrum

Unidades 1 2 3

Ian Romero

Kathleen Barrientos

Sergio Iporre

27 de octubre de 2026

Versión 1.3

Contenido

Resumen ejecutivo 3
Integración de las unidades 3
Calidad del proceso y requisitos 4
Organización del trabajo con Scrum 7
Plan de pruebas TP UF 01 9
Casos de prueba y trazabilidad 11
Prácticas orientadas a la calidad 13
Revisión y registro de hallazgos 14
Plan de acciones de mejora 15
Conclusiones y seguimiento 15
Referencias 16
Anexos de evidencia 17

Resumen ejecutivo

UniFood integra un catálogo universitario, carrito, pedidos y administración de productos en una aplicación web adaptable a dispositivos móviles. La primera implementación PWA incorpora instalación compatible y una pantalla sin conexión, mientras que las operaciones del negocio requieren acceso al servidor.

La verificación automatizada del 27 de octubre de 2026 registró 14 pruebas PHP aprobadas con 110 aserciones, 6 pruebas del service worker, 7 de Cypress y 4 de Selenium en GitHub Actions. Estos resultados respaldan los comportamientos incluidos en las suites; la cobertura de código, la instalación en teléfonos reales y los ensayos de carga todavía no se han medido.

El plan articula las tres unidades de la asignatura mediante requisitos verificables, historias de usuario, casos de prueba, trazabilidad y controles de calidad dentro de Scrum. Las responsabilidades fueron confirmadas por el usuario; fechas y estimaciones se acordarán en la planificación del equipo. Se adjuntan backlog, tablero GitHub y reportes reales de las cuatro suites, con campaña CI aprobada.

Se corrigió la recuperación de contraseña y se verificaron caducidad, hash, uso único y limitación de solicitudes. La entrega del enlace permanece configurada en log para la demostración; no se ha configurado correo de producción.

Control del documento	Registro
Código	SQA-UF-01
Versión y fecha	1.3 | 27 de octubre de 2026
Autores	Ian Romero, Kathleen Barrientos y Sergio Iporre
Destinatario	Evaluación académica de Software Quality Assurance
Objeto de verificación	Código verificado 14774aa01fc7d735df3a5dc9e6b65cdff87d1f32
Tipo de entrega	Informe académico y línea base del Test Plan

El informe integra los fundamentos de calidad, el diseño de pruebas y los controles de Scrum aplicados a UniFood. TP-UF-01 establece la primera línea base formal del plan de pruebas y conserva los resultados de verificación disponibles al cierre del informe.

Integración de las unidades

Los contenidos de las unidades se aplican al mismo producto y se relacionan con artefactos verificables. La actividad docente establece el Test Plan y las evidencias de la entrega [1]; los materiales de las unidades aportan los fundamentos y técnicas [2-4].

Unidad	Temas de los materiales	Aplicación a UniFood	Estado de la evidencia
Unidad 1	Conceptos y componentes SQA; calidad del proceso/producto; atributos y modelos; SDLC/STLC; requisitos; mejora continua	Requisitos y reglas de negocio; políticas de calidad; relación desarrollo-pruebas; riesgos y criterios medibles	Requisitos, políticas y relación SDLC/STLC documentados; validación de aceptación en el cierre académico

Unidad	Temas de los materiales	Aplicación a UniFood	Estado de la evidencia
Unidad 2	Git/GitHub/GitLab; Test Plan y casos; equivalencias y límites; caja negra/blanca; V&V; niveles de pruebas; herramientas de gestión y automatización; métricas y debugging	Repositorio, casos de prueba, matriz de trazabilidad, PHPUnit, pruebas JavaScript, selección de dos herramientas de la lista del plan	Campaña CI aprobada con PHPUnit, PWA, Cypress y Selenium; reportes XML, JSON y capturas conservados
Unidad 3	Scrum/Kanban/XP; inspecciones, revisiones y auditorías; TDD/ATDD/BDD/DDD	Backlog, objetivo de sprint, criterios de aceptación, DoR/DoD, registro de defectos, escenarios BDD y evidencia de corrección	Scrum elegido por el equipo. Proyecto GitHub con diez historias y PR vinculados; roles confirmados; eventos y estimaciones se registrarán por el equipo

La integración comprende calidad del proceso, herramientas SQA y prácticas de calidad durante el desarrollo ágil. La automatización y la integración continua se incorporan como requisitos de la actividad [1].

Calidad del proceso y requisitos

Descripción y objetivo del producto

UniFood permite a estudiantes consultar productos de la tienda universitaria, preparar un carrito y solicitar pedidos para pagar al recoger. El personal atiende los pedidos y el administrador mantiene el catálogo. La interfaz web se adapta a pantallas móviles y tiene una primera implementación PWA.

Actores del sistema: estudiante, empleado y administrador. Estos roles de acceso son distintos de las responsabilidades del equipo Scrum.

Requisitos funcionales

ID	Requisito verificable
RF-01	Permitir autenticación, cierre de sesión y acceso según rol
RF-02	Mostrar al estudiante productos activos y su disponibilidad
RF-03	Agregar, actualizar y quitar productos del carrito con totales calculados por el servidor
RF-04	Crear un pedido con identificador, detalle, total y estado pendiente; rechazar carrito vacío
RF-05	Permitir al estudiante consultar exclusivamente sus pedidos
RF-06	Permitir al personal avanzar pendiente → en preparación → listo → entregado; impedir modificar entregados
RF-07	Permitir al administrador crear y editar productos con nombre, categoría, precio y disponibilidad
RF-08	Permitir al administrador desactivar/reactivar productos conservando su registro
RF-09	Permitir cancelar un pedido propio únicamente mientras siga pendiente
RF-10	Ofrecer instalación PWA cuando el navegador la admita y mostrar una pantalla genérica al perder conexión

ID	Requisito verificable
RF-11	Permitir registro de estudiantes y recuperación de contraseña en el entorno de demostración; revisar sus controles de seguridad

Los requisitos se obtuvieron del análisis del código y del funcionamiento actual. Su validación corresponde al equipo y al Product Owner.

Reglas de negocio

- RN-01: solo el administrador modifica el catálogo; empleados y estudiantes reciben rechazo si intentan acceder a esas operaciones.
- RN-02: un pedido requiere al menos un producto válido; se comprueba nuevamente su actividad y disponibilidad al confirmar.
- RN-03: precios y totales proceden de la base de datos, no de precios enviados por el cliente.
- RN-04: el pedido entregado es inmutable; un pedido en preparación ya no puede cancelarse por el estudiante.
- RN-05: cada estudiante consulta/cancela únicamente pedidos propios.
- RN-06: Disponible y Activo son propiedades diferentes. Un producto agotado puede seguir visible; uno inactivo queda fuera del catálogo del estudiante.
- RN-07: la PWA requiere conexión para consultar datos y enviar pedidos. No hay cola de pedidos offline ni reenvío automático.

Criterios y políticas de calidad

Característica	Criterio de evaluación	Cómo observarlo
Funcionalidad	Los recorridos críticos cumplen sus criterios de aceptación	Casos y aserciones vinculados a historias
Seguridad y privacidad	Un estudiante no accede a pedidos ajenos; no se almacenan páginas privadas en el caché PWA	Pruebas de permisos y revisión de Cache Storage
Usabilidad	Flujos principales utilizables a 360 y 390 px; confirmaciones claras y operables con teclado	Exploración móvil, teclado, capturas y registro de hallazgos
Fiabilidad	Cancelar una confirmación no ejecuta la operación; un fallo de conexión no se presenta como éxito	Casos de cancelación y pérdida de red
Mantenibilidad	Cada defecto corregido tiene una prueba de regresión relevante	Revisión de cambios y suite automatizada
Compatibilidad	Validación en los navegadores y dispositivos declarados en el alcance	Matriz de navegador, versión y resultado real

Los modelos de Garvin, McCall e ISO/IEC 9126, junto con la referencia ISO/IEC 25010 presentada en clase, orientan el análisis académico. El apartado siguiente relaciona estos marcos con UniFood. Cada evaluación deberá identificar el modelo y la edición utilizados.

Políticas: revisar cambios por otro integrante cuando sea posible; registrar defectos; no publicar .env ni la base local; vincular requisitos con pruebas; documentar resultados reales; reejecutar regresiones después de cambios; revisar criterios antes de declarar una historia terminada.

Modelos de calidad aplicados

Los modelos trabajados en la unidad 1 permiten organizar preguntas sobre el proceso y el producto. En UniFood se emplean para definir condiciones observables y orientar la revisión [2].

Marco de referencia	Aplicación al proyecto	Evidencia esperada
Perspectivas de Garvin	Usuario: pedir sin confusión. Fabricante: cumplir requisitos. Producto: comportamiento comprobable. Valor: utilidad para la tienda y el estudiante.	Revisión de experiencia, criterios de aceptación y trazabilidad.
Factores de McCall	Corrección de totales, fiabilidad del pedido, integridad del acceso y facilidad de mantenimiento.	Pruebas de cálculo y permisos, regresión y revisión de cambios.
ISO IEC 9126	Analizar funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad conforme al marco estudiado.	Casos funcionales y no funcionales con resultados y limitaciones declarados.
Referencia ISO IEC 25010 de clase	Distinguir características del producto y del proceso, conservando la edición utilizada en los materiales.	Matriz de calidad con criterio, método de evaluación y evidencia.

Estos marcos se utilizan como referencias académicas. La aprobación de pruebas no constituye una certificación de conformidad con una norma.

SDLC y STLC aplicados

Desarrollo (SDLC)	Actividad de pruebas/calidad (STLC)	Evidencia
Requisitos	Revisar claridad, riesgos y condiciones de prueba	RF/RN y criterios de aceptación
Diseño	Diseñar escenarios, datos y permisos	Casos, matriz de roles, estados
Implementación	Pruebas de componentes y revisión de código	Pruebas y revisión de cambios
Integración	Ejecutar integración, sistema y regresión	Reporte de ejecución y defectos
Entrega	Aceptación y verificación de instalación/entorno	Checklist y demo revisada
Mantenimiento	Corregir hallazgos y repetir regresión	Registro de acciones y nueva evidencia

QA en UniFood: acordar políticas, revisar requisitos y prevenir fallos mediante criterios claros. QC: ejecutar casos y comprobar el comportamiento obtenido. Ejemplo de costo de calidad: diseñar una prueba de casilla es prevención/evaluación; corregir la casilla antes de entregar es una falla interna. La estimación económica requiere registrar tiempos y costos del equipo.

Organización del trabajo con Scrum

Distribución confirmada de responsabilidades

Integrante	Responsabilidad confirmada	Actividad concreta
Ian Romero	Product Owner y colaboración técnica	Priorizar backlog y validar criterios desde la necesidad de la tienda
Kathleen Barrientos	Scrum Master y colaboración en QA	Facilitar eventos, visibilizar impedimentos y coordinar revisión/evidencias
Sergio Iporre	Developer con foco en automatización	Mantener código, pruebas e integración continua
Los tres	Desarrollo y calidad compartidos	Revisar historias, ejecutar casos, corregir defectos y revisar resultados

El usuario confirmó esta distribución el 27 de octubre de 2026 en la conversación de preparación de la entrega. El registro confirma roles; no acredita asistencia a reuniones ni firmas individuales. Scrum define Product Owner, Scrum Master y Developers; QA es una responsabilidad compartida que puede ejercer un integrante con ese foco, no un cuarto rol formal separado. Si PO o Scrum Master realizan trabajo del incremento, también colaboran como Developers.

Objetivos y eventos

Objetivo del producto: permitir pedidos de comida del campus con catálogo administrable, privacidad por estudiante y seguimiento comprensible desde web/móvil.

Objetivo de la iteración: consolidar y demostrar el flujo estudiante-personal-administrador con reglas de negocio verificadas, trazabilidad y evidencias de calidad.

Duración, fecha de inicio, fecha de fin y capacidad: pendientes de acuerdo del equipo. La demostración académica se considera un hito de validación y no acredita por sí sola la ejecución de un sprint completo.

- Sprint Planning: elegir historias según valor/riesgo, acordar objetivo y dividir trabajo.
- Daily Scrum: inspeccionar avance e impedimentos y ajustar el trabajo del día.
- Sprint Review: demostrar el incremento y revisar criterios con interesados.
- Sprint Retrospective: acordar mejoras del proceso a partir de lo observado.

Los eventos se documentarán con participantes, fecha, decisiones y acciones. El trabajo implementado constituye la línea base de la iteración; las reuniones y estimaciones se registrarán a medida que se realicen.

Historias y Product Backlog inicial

Las historias especifican el comportamiento del producto existente y se someten a validación del Product Owner. Prioridad propuesta: P1 alta, P2 media. Puntos y asignación al sprint: por estimar en equipo.

ID y prioridad	Historia	Criterios de aceptación
HU-01 P1	Como usuario quiero iniciar/cerrar sesión para acceder a mis funciones	Credenciales válidas permiten entrar; credenciales inválidas no; acceso según rol; logout invalida sesión

ID y prioridad	Historia	Criterios de aceptación
HU-02 P1	Como estudiante quiero consultar el menú para elegir productos	Solo productos activos; precios visibles; agotados diferenciados
HU-03 P1	Como estudiante quiero gestionar mi carrito para revisar mi compra	Cantidades válidas; subtotales/total correctos; quitar producto actualiza el total
HU-04 P1	Como estudiante quiero confirmar mi pedido para recogerlo en la tienda	Se crea pedido pendiente con detalle; carrito vacío se rechaza; producto no disponible se rechaza
HU-05 P1	Como estudiante quiero consultar mis pedidos para conocer su estado	Solo pedidos propios; detalle coherente; acceso a pedido ajeno rechazado
HU-06 P1	Como empleado/administrador quiero avanzar pedidos para organizar la entrega	Transiciones permitidas en orden; entregado no se modifica
HU-07 P1	Como administrador quiero registrar/editar productos para mantener el menú	Precio positivo y nombre/categoría válidos; casilla Disponible se respeta; validación no guarda cambios parciales
HU-08 P1	Como administrador quiero retirar/reactivar productos para controlar su presencia	Desmarcar Activo lo retira; reactivarlo lo muestra; estudiantes/empleados no modifican catálogo
HU-09 P2	Como estudiante quiero cancelar un pedido pendiente para corregir una solicitud	Solo propio y pendiente; Volver/Escape no ejecutan cancelación; confirmar solicita una sola operación
HU-10 P2	Como usuario móvil quiero instalar UniFood y recibir un aviso sin conexión	Manifiesto/iconos válidos; instalación en entorno compatible; fallo de red muestra aviso sin datos privados

RF-11 incorpora recuperación de contraseña. Se corrigió el defecto DEF-03 y se verificaron vigencia, hash, uso único y limitación de solicitudes mediante PasswordRecoveryTest. La entrega del enlace sigue destinada a la demostración local en log.

Sprint Backlog sugerido para preparar la demostración

1. Validar RF/RN y criterios de HU-04, HU-05, HU-07 y HU-08.
2. Asignar responsables y revisar este Test Plan.
3. Preparar datos aislados y diseñar casos, incluidos errores y permisos.
4. Ejecutar la suite existente y guardar logs con versión del código.
5. Ejecutar el recorrido manual y registrar defectos/capturas.
6. Conservar la campaña aprobada de Cypress y Selenium y sus reportes vinculados.
7. Revisar una historia, inspeccionar su código y completar un registro de auditoría.
8. Revisar resultados y ordenar lo pendiente para el siguiente incremento.

Tablero publicado: https://github.com/users/kat506/projects/1. Diez historias HU-01 a HU-10 enlazan a los issues 2 a 11 de ifrancro/UniFood. El PR 1 reúne los cambios técnicos; DEF-03 se registra en el issue 12. El CSV permite conservar una copia local. Flujo: Por validar → En curso → En revisión → Terminado. Vincular en cada tarjeta HU, RF, casos, defectos, evidencia y commit. Un tablero con esas columnas apoya Scrum, pero por sí solo no demuestra que se haya aplicado todo el marco.

Criterios de entrada y salida

Criterios de preparación de las historias: historia comprensible; criterios verificables; dependencias y riesgos conocidos; datos/entorno definidos; prioridad acordada. No es un elemento formal obligatorio de Scrum.

Criterios de terminación: aceptación satisfecha; cambio revisado; regresión aprobada; defectos críticos resueltos o fuera del alcance acordado; evidencia y documentación actualizadas; incremento integrado y usable.

Plan de pruebas TP UF 01

Introducción

El plan TP-UF-01 establece el alcance, la estrategia y los controles de calidad del catálogo y del recorrido de pedidos dentro de Scrum. Constituye la primera línea base documental del proyecto y servirá como referencia para evaluar resultados y registrar futuras actualizaciones.

Control de cambios del plan y su evidencia:

Versión	Cambio documentado	Evidencia de cierre
1.1	Primera integración formal de unidades y resultados PHP/PWA	11 pruebas PHP, 94 aserciones y seis del worker
1.2	Selección de Cypress/Selenium y workflow; registro de bloqueos	Scripts y reportes de intento local
1.3	Roles confirmados, tablero publicado, recuperación corregida y campaña CI aprobada	14 PHP, seis PWA, siete Cypress y cuatro Selenium; XML/JSON y capturas

Alcance

Incluidos: autenticación y permisos; catálogo; carrito; creación/consulta/cancelación de pedidos; cambios de estado; creación/edición/desactivación/reactivación de productos; confirmaciones; primera PWA y vista móvil.

Fuera de la funcionalidad actual: pago online, entrega a domicilio, aplicación nativa/APK, notificaciones push y envío offline de pedidos. Recuperación de contraseña: entrega de enlaces en log para demostración; controles de caducidad, hash, uso único y limitación de solicitudes verificados. El correo de producción no forma parte de esta entrega.

Estrategia y herramientas

- Revisión estática: requisitos, formularios, controladores, reglas de permisos y pruebas.
- Integración/backend: PHPUnit con SQLite en memoria para aislar datos de prueba.
- Componentes PWA: pruebas JavaScript del worker con entorno simulado; no equivalen a instalación real en teléfono.
- Sistema/aceptación: recorridos completos desde navegador con cuentas de cada rol.
- Regresión: repetir pruebas críticas tras corregir defectos.
- Exploración/no funcionales: teclado, tamaños móviles, desconexión, privacidad y compatibilidad.
- Rendimiento/estrés: diseñar objetivos y carga medibles antes de ejecutar; aún no realizados.

Herramientas utilizadas: Git/GitHub para versionado y seguimiento, PHPUnit para backend, Node test runner para el worker, Cypress y Selenium para navegador y Vite para compilación.

Dos herramientas de la lista del PDF: Cypress 16.2.0 y Selenium WebDriver 4.51.0. Cypress reúne siete casos de autenticación, permisos, confirmación, carrito y vista móvil. Selenium ejecuta cuatro verificaciones de login, carrito, confirmación y adaptación móvil en Chrome real. Usar ambas permite comparar la ejecución integrada de Cypress con el control mediante WebDriver; el catálogo de casos

conserva la trazabilidad común. La gestión del backlog se documenta por separado y no se atribuye a estas herramientas.

La campaña final en Ubuntu de GitHub Actions aprobó 14 pruebas PHP con 110 aserciones, seis pruebas del worker, siete pruebas de Cypress y cuatro de Selenium. Código verificado: 14774aa01fc7d735df3a5dc9e6b65cdff87d1f32. Ejecución: https://github.com/ifrancro/UniFood/actions/runs/40628543184. Cada suite conserva su resultado; las 31 pruebas ejecutadas no equivalen a cobertura del 100 por ciento del producto.

Los intentos iniciales permitieron corregir dos problemas de la automatización: instalación explícita del binario Cypress debido a ignore-scripts y aislamiento de cada caso Selenium en una sesión nueva. Los fallos iniciales y el cierre aprobado se conservan en el historial del PR. El bloqueo del navegador local se superó ejecutando la campaña en CI; la prueba de instalación en un teléfono físico sigue fuera de esta evidencia.

CI instala PHP 8.4 y Node 22, compila la interfaz, prepara una base SQLite exclusiva, ejecuta las cuatro suites y conserva XML JUnit, JSON y capturas como artefactos. La configuración quedó publicada en .github/workflows/calidad.yml dentro del PR 1. Su resultado final está aprobado y es verificable en el enlace anterior.

Roles

La distribución de responsabilidades se presenta en el apartado de organización con Scrum. Las revisiones e inspecciones se asignarán, cuando sea posible, a un integrante distinto del autor del cambio. Cada ejecución identificará a su responsable.

Ciclo de vida de pruebas en el sprint

Refinamiento: analizar riesgos y acordar criterios. Planning: seleccionar casos e identificar automatización. Implementación: diseñar/ejecutar pruebas y revisar cambios. Integración: ejecutar regresión y corregir. Review: demostrar resultados y recoger comentarios. Retrospectiva: proponer mejoras basadas en fallos, demoras y claridad de requisitos.

Entrada, salida y suspensión

Entrada: código identificable por commit; dependencias compatibles; entorno separado de datos que se quieran conservar; cuentas y datos de prueba; criterios y casos listos.

Salida propuesta: casos críticos ejecutados y aprobados; regresión aprobada; hallazgos registrados y atendidos; evidencias adjuntas; limitaciones declaradas; revisión de aceptación pendiente o realizada con nombre y fecha de revisión.

Suspender ante entorno corrupto, versión PHP incompatible, datos de prueba no aislados o error que impida iniciar sesión. Corregir el impedimento y reejecutar los casos afectados. No borrar la base de trabajo para intentar resolver un problema de pruebas.

Métricas y control

Métrica	Cálculo/registro	Situación
Tasa de aprobación	Aprobados / ejecutados × 100, por suite y ejecución	Ejecución del 27 de octubre de 2026: 14/14 pruebas aprobadas y 110 aserciones. Commit 14774aa01fc7
Resultado PWA	Aprobados/ejecutados, separado de PHPUnit	Ejecución del 27 de octubre de 2026: 6/6 pruebas aprobadas en entorno simulado

Métrica	Cálculo/registro	Situación
Resultado de navegador	Aprobados / ejecutados por herramienta	Cypress 7/7 y Selenium 4/4; campaña CI 40628543184 aprobada
Cobertura de requisitos	RF con al menos un caso ejecutado / RF incluidos	Pendiente de completar resultados y trazabilidad
Cobertura de código	Sentencias/ramas ejecutadas con instrumentación	No medida; aprobar todas las pruebas no equivale a 100% de cobertura
Defectos	Abiertos/cerrados/reabiertos por severidad, fecha y versión	Tres defectos corregidos: dos de casillas y uno de tokens; regresión aprobada
Mejora del proceso	Acciones de retrospectiva completadas / acordadas	Pendiente: aún no se registró retrospectiva
Velocidad	Puntos terminados por sprint con un mismo criterio de estimación	No disponible; no hay sprints previos documentados

Las tasas de aprobación se interpretan por suite y versión. La cobertura global del producto requiere medir requisitos y código por separado.

Riesgos y mitigación

Riesgo	Tratamiento propuesto
PHP incompatible con composer.lock	Documentar PHP >=8.4.1 y verificar requisitos de plataforma
Casos que afecten datos reales	SQLite en memoria para automatización; datos de demo identificados para manuales
Falta de trazabilidad	Asignar IDs RF/HU/CP/DEF y enlazar resultados y commit
PWA probada solo en emulación	Ejecutar instalación con HTTPS en teléfono real y registrar dispositivo/navegador
Poca cobertura negativa y de concurrencia	Añadir datos inválidos, permisos y operaciones simultáneas según riesgo
Entrega de enlaces de recuperación en entorno demo	Los controles del token están verificados; mantener el log privado y configurar correo antes de uso operativo
Regresión de automatización	Cypress y Selenium ya ejecutados; repetir CI al integrar cambios y conservar artefactos

Casos de prueba y trazabilidad

El catálogo reúne 17 casos de diseño. Las suites ejecutadas verifican los comportamientos identificados en la matriz; algunos casos amplios tienen comprobación parcial. El informe no atribuye aprobación completa a un caso solo por contar con una prueba relacionada. La instalación física, exploración de teclado, carga y cobertura instrumentada requieren campañas adicionales.

Caso y trazabilidad	Procedimiento y técnica	Resultado esperado
CP-01 RF-01 / HU-01	Cuenta válida; iniciar sesión con datos correctos y luego incorrectos Técnica: Equivalencias	Acceso correcto según rol; datos incorrectos rechazados
CP-02 RF-07/08 / HU-07/08	Entrar como estudiante y empleado; intentar crear/editar/desactivar producto Técnica: Tabla de decisión por rol	Operaciones rechazadas y producto sin cambios
CP-03 RF-02 / HU-02	Producto activo y producto inactivo; abrir catálogo estudiante Técnica: Tabla de decisión	Solo activo aparece
CP-04 RF-03 / HU-03	Agregar cantidad 0, 1 y 2; luego actualizar a 0 Técnica: Límites y equivalencias	Agregar 0 se rechaza; 1 y 2 válidos; actualizar a 0 elimina la línea
CP-05 RF-03/04 / HU-03/04	Producto de 8 Bs.; agregar 2 y confirmar Técnica: Integración	Total 16 Bs.; pedido con 2 unidades y estado pendiente
CP-06 RF-04 / HU-04	Carrito vacío; intentar confirmar Técnica: Caso negativo	Sin pedido creado y mensaje claro
CP-07 RF-04 / HU-04	Agregar producto; personal lo marca agotado antes de confirmar Técnica: Integración / reglas	No se confirma con producto no disponible; mensaje y carrito coherentes
CP-08 RF-05 / HU-05	Dos estudiantes; intentar acceder al pedido del otro Técnica: Seguridad funcional	Acceso rechazado y sin datos ajenos
CP-09 RF-06 / HU-06	Personal; avanzar pedido por sus estados; repetir al entregarlo Técnica: Transición de estados	Orden permitido; entregado no se modifica
CP-10 RF-07 / HU-07	Crear producto con Disponible marcado y desmarcado Técnica: Tabla de decisión booleana	Se guardan true y false respectivamente
CP-11 RF-07 / HU-07	Precio 0.00, 0.01 y 0.02 Técnica: Valores límite	0.00 rechazado; 0.01 y 0.02 válidos
CP-12 RF-08 / HU-08	Producto activo; desmarcar Activo, guardar, revisar catálogo; reactivar Técnica: Transición de estado	Se retira y luego reaparece; registro conservado
CP-13 RF-08 / HU-08	Abrir Desactivar; elegir Volver o Escape; luego repetir y confirmar sobre producto de prueba Técnica: Interfaz / aceptación	Cancelar no modifica; confirmar desactiva una sola vez
CP-14 RF-09 / HU-09	Pedido propio pendiente y otro en preparación; intentar cancelarlos Técnica: Tabla de decisión / estados	Solo pendiente puede cancelarse; cancelación de confirmación no opera
CP-15 RF-10 / HU-10	Primera visita online; instalación en HTTPS y posterior navegación sin red Técnica: Compatibilidad / instalación	Instalación compatible y pantalla offline sin datos privados
CP-16 RF-01 a RF-10 / varias	Recorridos principales a 360/390 px y con teclado Técnica: Exploratoria / usabilidad	Controles utilizables, foco visible y sin desbordamientos que bloqueen el flujo

Caso y trazabilidad	Procedimiento y técnica	Resultado esperado
CP-17 RF-11 / pendiente	Registrar usuario; usar recuperación; volver a usar token y comprobar su antigüedad Técnica: Equivalencias / seguridad; aceptación pendiente	Registro como estudiante; política de vigencia y uso único definida y verificada

Para CP-13 usar producto de prueba, no retirar productos reales del usuario sin necesidad. Para CP-15 la emulación del navegador no sustituye la instalación en Android/iOS.

Plantilla de ejecución: CP | commit/versión | entorno/navegador | datos usados | pasos ejecutados | resultado esperado | resultado obtenido | aprobado/fallido/bloqueado | evidencia | defecto | ejecutor | fecha de ejecución.

Matriz de trazabilidad inicial

Requisitos	Historias	Casos	Automatización relacionada existente
RF-01	HU-01	CP-01/02	UniFoodExperienceTest y Cypress; login válido/incorrecto y permisos; ampliar logout
RF-02/03/04	HU-02/03/04	CP-03 a CP-07	UniFoodExperienceTest; ampliar límites y cambios de disponibilidad durante compra
RF-05/06	HU-05/06	CP-08/09	UniFoodExperienceTest
RF-07/08	HU-07/08	CP-02/10/11/12/13	AdminProductosTest; Cypress y Selenium verifican apertura y Volver de CP-13; ampliar confirmación y Escape
RF-09	HU-09	CP-14	Ampliación pendiente para cancelación y confirmación
RF-10	HU-10	CP-15/16	Pruebas del worker, Cypress y Selenium a 390 px; instalación física y teclado no verificados
RF-11	Corrección DEF-03	CP-17	PasswordRecoveryTest: hash, vencimiento, uso único y throttle; registro de estudiante requiere ampliar campaña

La suite PHP contiene principalmente pruebas funcionales y de integración con Laravel. La cobertura de caja blanca requerirá instrumentación de sentencias y ramas; actualmente no se dispone de esa medición.

Prácticas orientadas a la calidad

Ejemplo de escenario BDD propuesto

```
Característica: Control del catálogo
  Escenario: Desactivar un producto desde su edición
    Dado que un administrador edita un producto activo
    Cuando desmarca Activo y guarda cambios válidos
    Entonces el producto queda inactivo
    Y el estudiante deja de verlo en el catálogo
    Y se conserva el registro para poder reactivarlo
```

Segundo criterio de aceptación propuesto para la misma historia: al volver a marcar Activo y guardar, el producto reaparece para el estudiante. Acordar ambos criterios con quien ejerza PO antes del próximo cambio para aplicar ATDD. La redacción Gherkin todavía no implica que exista un ejecutor BDD configurado.

Corrección guiada por pruebas ya observada

Se añadieron pruebas para las casillas de administración, se ejecutaron contra el comportamiento anterior y se observaron dos fallos. Se corrigió el guardado de Disponible/Activo y después la suite PHP pasó con 11 pruebas y 94 aserciones.

Esto sirve como ejemplo de regresión con un ciclo rojo-verde durante la corrección de defectos. No prueba que todo UniFood haya sido desarrollado con TDD, ni que se completara/documentara una fase específica de refactorización. Los resultados se conservan en el anexo de verificación automatizada. El estado corregido se relaciona con el commit revisado.

Dominio y DDD

Glosario inicial: Producto (actividad, disponibilidad, categoría y precio); Carrito (selección temporal de productos/cantidades); Pedido (solicitud con detalles, importe y estado); DetallePedido (copia de nombre, precio y cantidad); Usuario (cuenta con rol).

Usar ese lenguaje consistente en historias, código y pruebas. Identificar entidades es una introducción al modelado del dominio, no evidencia de una implantación completa de DDD ni motivo suficiente para rehacer la arquitectura antes de la entrega.

Revisión y registro de hallazgos

Este registro recoge observaciones técnicas y brechas documentales. No constituye una auditoría independiente final; debe ser revisado por otro integrante con fecha y evidencia.

ID	Hallazgo	Acción/estado	Evidencia disponible y pendiente
DEF-01	Crear producto con Disponible desmarcado podía guardarlo disponible	Corregido y verificado mediante regresión automatizada	AdminProductosTest y registro de ejecución del anexo
DEF-02	Desmarcar Activo al editar no retiraba el producto	Corregido; prueba de retiro/reactivación	AdminProductosTest y verificación de retiro y reactivación del catálogo
OBS-01	Ausencia inicial de Test Plan y backlog documental	Línea base documental elaborada en esta entrega	Este informe
OBS-02	Faltaba ejecución de dos herramientas y CI	Cerrado técnicamente: Cypress 7/7 y Selenium 4/4; CI aprobado	Ejecución 40628543184 y artefactos XML, JSON y capturas
DEF-03 antes OBS-03	Recuperación sin comprobación de edad del token	Corregido con broker de Laravel: hash, vencimiento, uso único y throttle	PasswordRecoveryTest, tres regresiones; issue 12 cerrado con evidencia CI
OBS-04	Instalación PWA en teléfono real sin evidencia	Pendiente	Registrar dispositivo, navegador, HTTPS y capturas
OBS-05	Falta de registros de ejecución de los eventos Scrum	Formalización y ejecución planificadas	Roles confirmados y formato para eventos reales

Checklist para revisión/inspección: criterios inequívocos; roles autorizados; validación en servidor; totales correctos; transiciones permitidas; respeto de casillas; casos negativos; protección de datos; resultados trazables; claridad de mensajes; evidencia repetible.

Formato de auditoría propuesto: objetivo | alcance | criterio | evidencia examinada | responsable real | hallazgo | severidad | acción | fecha límite | responsable de acción | verificación de cierre. Concluir sobre lo comprobado y mantener pendientes explícitos.

Plan de acciones de mejora

1. Acordar roles, alcance, fecha y objetivo de la iteración; revisar RF/RN y casos críticos.
2. Validar la línea base y establecer el tablero con historias, casos y evidencias vinculadas.
3. Conservar la campaña CI aprobada y repetirla ante nuevos cambios; revisar artefactos y enlazarlos a los casos correspondientes.
4. Ejecutar la suite de la versión que se presentará y guardar reportes con commit, entorno y fecha.
5. Probar manualmente estudiante → pedido → empleado → entrega y administrador → producto → catálogo.
6. Ejecutar negativos: carrito vacío, producto agotado, pedido ajeno, permisos y casillas desmarcadas.
7. Registrar revisión/inspección de HU-07 y HU-08 usando DEF-01/02; completar auditoría con otro integrante.
8. Actualizar métricas reales y preparar una exposición breve de proceso, herramientas, resultados y mejoras.

La verificación realizada respalda las funciones incluidas en las suites de pedidos, permisos, administración y caché PWA. La trazabilidad y los criterios definidos establecen una base para ampliar la evaluación sin confundir aprobación de pruebas con cobertura total.

Conclusiones y seguimiento

La entrega integra requisitos y modelos de calidad de la unidad 1; técnicas, Test Plan y herramientas de la unidad 2; y organización Scrum y prácticas de calidad de la unidad 3. Se dispone de campaña automatizada reproducible y aprobada, backlog publicado y evidencia de tres correcciones. Los roles están confirmados por el usuario.

Los límites de la conclusión son la cobertura parcial de casos amplios, ausencia de medición de ramas/sentencias, instalación física no verificada y ausencia de actas de eventos celebrados. El equipo completará lo siguiente con información real:

- Fechas y capacidad de la iteración, estimación y validación de criterios con Product Owner.
- Versión revisada/aprobada de TP-UF-01 y comparación con su próxima versión.
- Seguimiento del tablero ya publicado y actualización de estados al validar las historias.
- Revisión de los XML, JSON y capturas exportados de la campaña aprobada.
- Demostración del pipeline aprobado y explicación de los fallos iniciales corregidos.
- Auditoría interna revisada y acciones con responsables.
- Revisión de aceptación, eventos Scrum y mejoras del siguiente incremento con participantes y fechas efectivas.

Referencias

[1] Universidad del Valle. Plan SQA web. Actividad grupal Actualización del Test Plan en un entorno ágil. Material docente facilitado para la asignatura.

[2] Universidad del Valle. Software Quality Assurance. Clases I a VIII y dinámica El código que llegó demasiado lejos. Material de la unidad 1.

[3] Universidad del Valle. Software Quality Assurance. Clases IX a XV. Material de herramientas SQA, diseño, técnicas y niveles de pruebas.

[4] Universidad del Valle. Software Quality Assurance. Clases XVI a XVIII. Material sobre desarrollo ágil, controles de calidad y prácticas TDD, BDD, ATDD y DDD.

[5] Schwaber, K. y Sutherland, J. (2020). The Scrum Guide. https://scrumguides.org/scrum-guide.html

[6] UniFood. Código fuente local, documentación técnica y suites de prueba. Commit revisado 1e1a2b3cad87e4ea994f1faf9fa5fd0760cbd1ef. Repositorio configurado https://github.com/ifrancro/UniFood. Consulta técnica 27 de octubre de 2026.

[7] UniFood. Campaña GitHub Actions 40628543184, artefacto evidencias-calidad 12790158653. Reportes y capturas del 27 de octubre de 2026, hora de Bolivia. https://github.com/ifrancro/UniFood/actions/runs/40628543184

Las especificaciones de negocio, el diseño de casos y las prioridades de esta versión requieren validación del equipo. Las referencias [1-4] fundamentan la estructura académica y [6-7] aportan la evidencia técnica examinada.

Anexos de evidencia

Campaña final y seguimiento

Fecha: 27 de octubre de 2026, hora de Bolivia. GitHub Actions, Ubuntu y Chrome. El servidor utilizó SQLite exclusivo para E2E. La base de la demostración se conservó. Código verificado: 14774aa01fc7d735df3a5dc9e6b65cdff87d1f32.

Ejecución aprobada: https://github.com/ifrancro/UniFood/actions/runs/40628543184. Artefacto evidencias-calidad, identificador 12790158653. Se conservaron reportes XML JUnit, resultado JSON de Selenium y capturas de ambas herramientas.

Herramienta	Resultado	Evidencia
Cypress 16.2.0	7 de 7 aprobados	XML JUnit y capturas de administración y catálogo móvil.
Selenium 4.51.0	4 de 4 aprobados	JSON y capturas de confirmación y catálogo móvil.
GitHub Actions	Todos los pasos aprobados	Ejecución 40628543184 sobre 14774aa01fc7.

Tablero público: https://github.com/users/kat506/projects/1. Las diez historias están vinculadas a los issues 2 a 11 del repositorio, con criterios de aceptación y casos. El PR 1 reúne los cambios y DEF-03 se cerró en el issue 12. Los roles están confirmados; las reuniones y aceptaciones del equipo se registrarán cuando efectivamente ocurran.

Verificación automatizada

Verificación local adicional del 27 de octubre de 2026: PHP 8.4.27, PHPUnit 12.5.34 y SQLite en memoria. Se precargó el autoload por una limitación de lectura del entorno; se conservaron las pruebas y sus resultados.

Suite	Resultado	Alcance de la conclusión
PHPUnit	14 de 14 aprobadas 110 aserciones	Comprueba los comportamientos definidos en las pruebas PHP existentes, incluidas las regresiones del administrador.
Service worker	6 de 6 aprobadas	Comprueba navegación y aislamiento del caché en un entorno simulado; no acredita instalación en un teléfono.

Extracto de PHPUnit

```
PHPUnit 12.5.34 by Sebastian Bergmann and contributors.
Runtime:       PHP 8.4.27
..............                                                  14 / 14 (100%)
Time: 01:21.213, Memory: 50.00 MB
OK (14 tests, 110 assertions)
```

Extracto de las pruebas PWA

```
# tests 6
# pass 6
# fail 0
```

Las suites ejecutadas aprobaron todos sus casos. No se midió cobertura completa de sentencias, ramas ni requisitos.

Confirmación integrada en UniFood

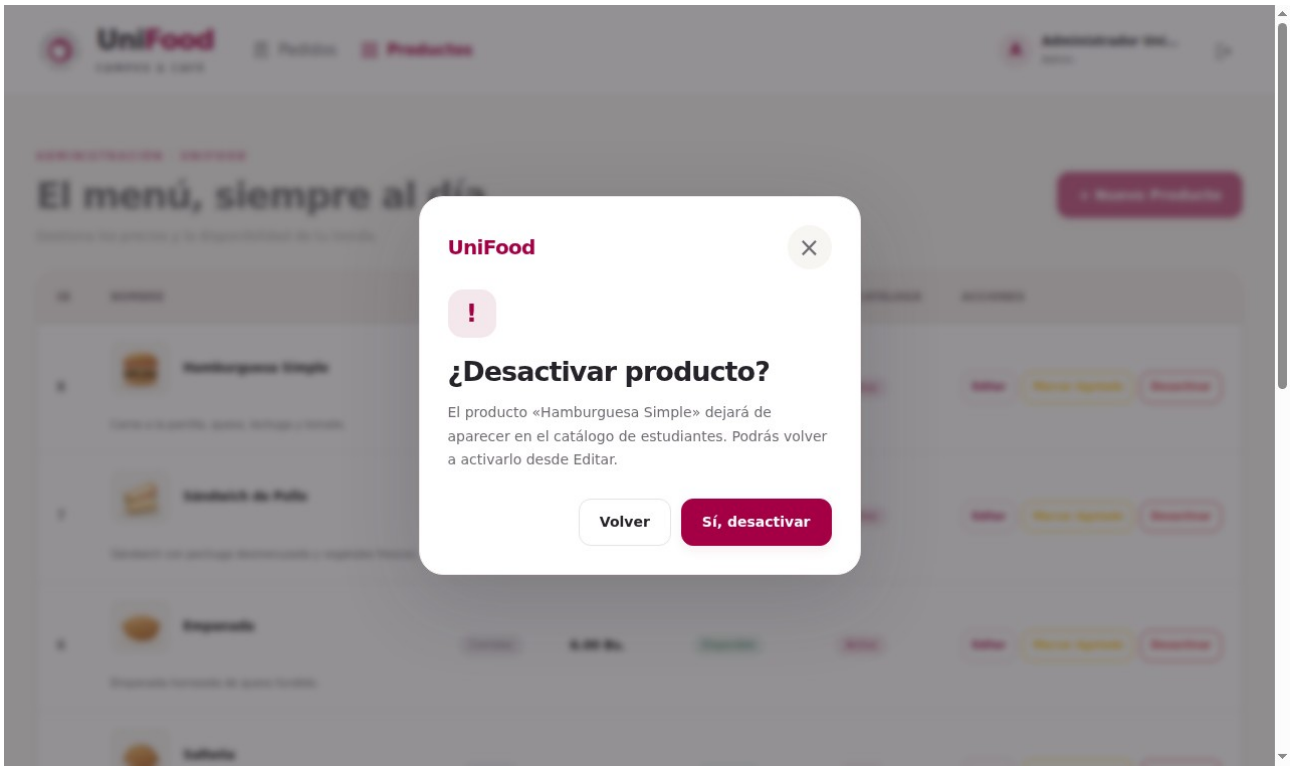


Figura 1. Confirmación propia de UniFood capturada durante Selenium en CI. La prueba abre el diálogo, captura el estado y verifica que Volver lo cierra. Las correcciones de las casillas se verifican mediante PHPUnit; este caso no acredita por sí mismo confirmar la desactivación.

Fuente: artefacto de la campaña automatizada de UniFood [7].

Catálogo en vista móvil

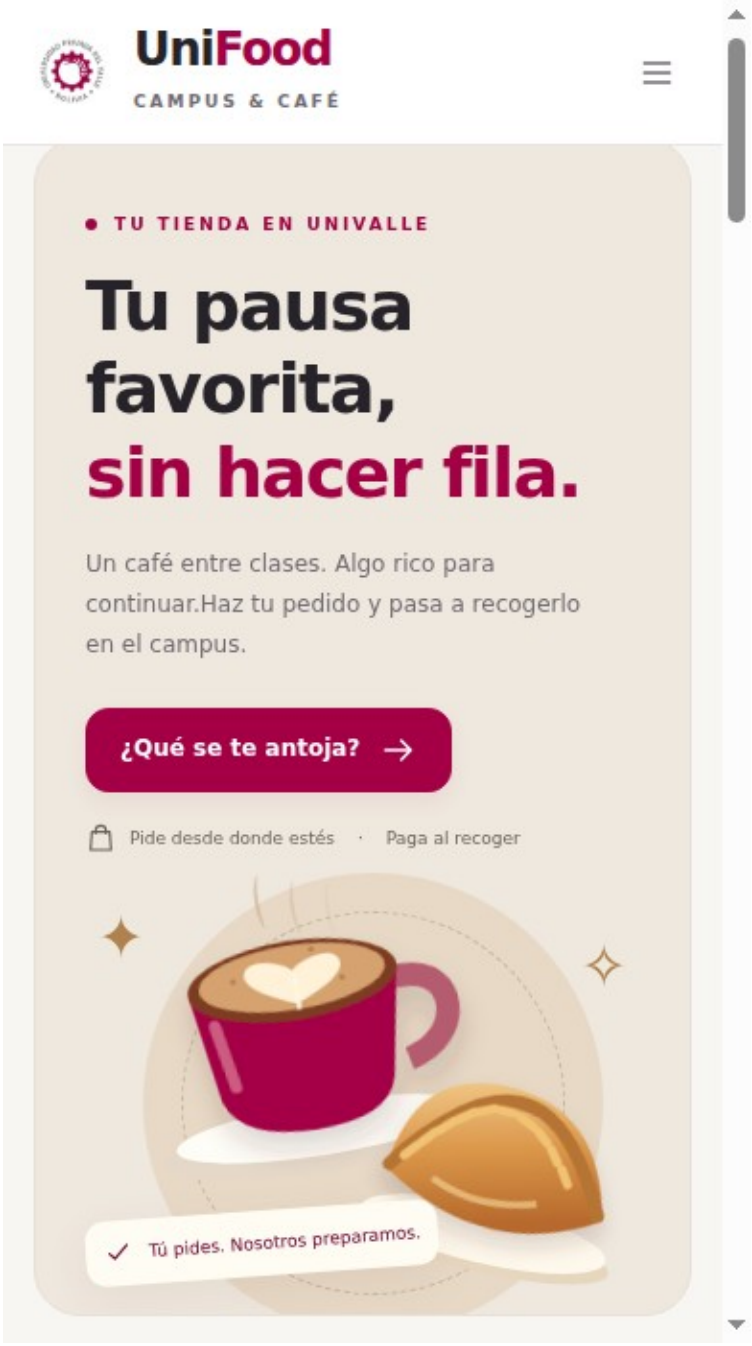


Figura 2. Catálogo en una ventana de Chrome configurada a 390 por 844 píxeles mediante Selenium. Campaña CI aprobada del código 14774aa01fc7. Acredita la vista móvil comprobada; no equivale a instalar la PWA en un teléfono físico.

Fuente: artefacto de la campaña automatizada de UniFood [7].

Ficha de ejecución de pruebas

La siguiente campaña debe relacionar cada resultado con el caso de prueba y el código presentado. El siguiente formato se utilizará al ejecutar los casos; los campos se completarán con datos reales.

Campo	Dato a registrar
Identificación	Caso CP y requisito RF o historia HU asociados.
Versión	Commit y fecha de la ejecución.
Entorno	PHP, base de prueba, navegador, versión y dispositivo.
Responsable	Integrante que ejecutó el caso y quien revisó el resultado.
Procedimiento	Datos utilizados y pasos efectivamente ejecutados.
Resultado	Esperado, obtenido y estado aprobado, fallido o bloqueado.
Evidencia	Archivo de captura o reporte y defecto relacionado.
Seguimiento	Acción correctiva, responsable y verificación de cierre.

La campaña, el tablero y la corrección de recuperación cuentan con evidencia conservada. La siguiente revisión del equipo debe registrar aceptación, eventos reales y comprobaciones que amplíen la cobertura, incluida instalación en un teléfono. Los campos de este formato se completarán durante esa ejecución, sin atribuir resultados no observados.